Aritmética modular y criptografía RSA

Librería de teoría de números, intérprete IMAT-LAB y criptosistema RSA con fines didácticos

Autores Jaime Tuesta de Simón
Ignacio Mascarell Agúndez

Grupo GP11B

Titulación Grado en Ingeniería Matemática e Inteligencia Artificial

Centro ICAI, Universidad Pontificia Comillas

Curso 2025–2026

Índice

1 Introducción **2**

2 Librería de aritmética modular **2**
2.1 Resumen de funciones . . . 2
2.2 Primalidad y factorización . . . 3
2.3 Máximo común divisor e identidad de Bézout . . . 3
2.4 Potencias, inversas y función de Euler . . . 3
2.5 Símbolo de Legendre . . . 4
2.6 Sistemas de congruencias . . . 4
2.7 Raíces cuadradas: algoritmo de Cipolla . . . 4
2.8 Ecuaciones de segundo grado . . . 5

3 Intérprete IMAT-LAB **5**

4 Criptosistema RSA **6**
4.1 Generación de claves . . . 6
4.2 Padding . . . 6
4.3 Cifrado de cadenas . . . 6

5 Programas de usuario **7**
5.1 Registro de usuarios . . . 7
5.2 Criptochat . . . 7

6 Ataques a RSA **7**
6.1 Ataque contra la clave . . . 7
6.2 Ataque de texto claro elegido . . . 8

7 Limitaciones **8**

8 Bibliografía **8**

1 Introducción

El trabajo se organiza en cinco ficheros de Python, sin dependencias externas:

Fichero	Contenido
`modular.py`	Librería de aritmética modular (práctica 1).
`imatlab.py`	Intérprete IMAT-LAB: acceso interactivo o por lotes a `modular.py` (práctica 1).
`rsa.py`	Funciones de RSA: claves, padding, cifrado de números y cadenas y ataques (práctica 2).
`registrarusuario.py`	Programa que crea un usuario y guarda sus claves (práctica 2).
`criptochat.py`	Chat cifrado entre dos usuarios registrados (práctica 2).

La memoria sigue ese orden: primero la librería y las decisiones matemáticas que hay detrás de cada función, después IMAT-LAB y, por último, el criptosistema RSA, los programas de usuario y los ataques.

En toda la librería se busca que el código sea legible para alguien externo: cada función documenta qué recibe, qué devuelve y qué excepciones lanza, y se comentan los pasos que no son evidentes. Las funciones señalan los casos sin solución con dos excepciones: `ZeroDivisionError` (por ejemplo, una inversa que no existe) e `IncompatibleEquationError` (ecuaciones o sistemas sin solución).

2 Librería de aritmética modular

2.1 Resumen de funciones

Función	Algoritmo
`es_primo(n)`	División por 2, por 3 y por los candidatos $6k \pm 1$ hasta $\sqrt{n}$.
`lista_primos(a,b)`	Recorre $[a, b)$ y aplica `es_primo`.
`factorizar(n)`	Extrae los factores 2 y 3 y prueba los divisores $6k \pm 1$ mientras $p^2 \leq n$.
`mcd(a,b)`	Algoritmo de Euclides iterativo.
`bezout(n,m)`	Euclides extendido, con el algoritmo de Lehmer para números grandes.
`mcd_n, bezout_n`	Extensión a listas mediante la asociatividad del mcd.
`coprimos(a,b)`	Comprueba si $\mathrm{mcd}(a, b) = 1$.
`potencia_mod_p`	Exponenciación binaria; exponentes negativos mediante la inversa.
`inversa_mod_p`	Coeficiente de Bézout reducido módulo p.
`euler(n)`	Fórmula del producto a partir de la factorización.
`legendre(n,p)`	Criterio de Euler: $n^{(p-1)/2} \bmod p$.
`resolver_sistema_congruencias`	Reducción de cada ecuación y combinación sucesiva de congruencias.
`raiz_mod_p(n,p)`	Algoritmo de Cipolla.
`ecuacion_cuadratica`	Fórmula cuadrática con inversas y raíces modulares.

2.2 Primalidad y factorización

Todo primo mayor que 3 es de la forma $6k \pm 1$, ya que los restos 0, 2, 3 y 4 módulo 6 corresponden a múltiplos de 2 o de 3. Además, si n es compuesto tiene un divisor primo $p \leq \sqrt{n}$. Por eso `es_primo` descarta primero los múltiplos de 2 y 3 y después solo prueba los candidatos $6k-1$ y $6k+1$ hasta $\sqrt{n}$, lo que reduce las divisiones a un tercio. El test es determinista.

`lista_primos(a,b)` devuelve los primos del intervalo $[a, b)$, ajustando el extremo inferior a 2. Si $b < a$ la lista es vacía y, como caso especial, `lista_primos(a,a)` devuelve $[a]$ cuando a es primo.

`factorizar` utiliza la misma rueda: divide todas las veces posibles por 2 y por 3 y después por los candidatos $6k \pm 1$ mientras $p^2 \leq n$. Si al terminar queda un factor mayor que 1, ese factor es primo. El resultado es un diccionario `{primo: exponente}`, único por el teorema fundamental de la aritmética.

La librería incluye también una implementación del test probabilístico de Miller–Rabin (`miller_rabin_algorithm`, con 10 bases aleatorias y probabilidad de error menor que 4^{-10}). Es una alternativa que no se utiliza en el resto de la librería: todas las funciones emplean el test determinista de `es_primo`.

2.3 Máximo común divisor e identidad de Bézout

`mcd` aplica el algoritmo de Euclides de forma iterativa, basado en $\gcd(a, b) = \gcd(b,\ a \bmod b)$, sobre los valores absolutos de las entradas.

`bezout(n,m)` devuelve (d, x, y) con $d = \gcd(n, m) = nx + my$. Los coeficientes se calculan para $|n|$ y $|m|$ manteniendo una matriz 2×2 con la combinación lineal de cada resto, y al final se ajusta su signo según el de las entradas. Cuando ambos números superan los 128 bits se aplica el algoritmo de Lehmer: con los 128 bits más significativos se calculan, en aritmética de tamaño fijo, varios cocientes consecutivos que coinciden con los de los números completos, y se aplican todos a la vez como una única matriz. Así se evitan muchas divisiones entre números grandes. Cuando ya no es seguro calcular más cocientes de esta forma, se continúa con Euclides estándar.

`mcd_n` y `bezout_n` extienden ambas funciones a una lista mediante $\gcd(a_1, \ldots, a_n) = \gcd(\gcd(a_1, \ldots, a_{n-1}), a_n)$: en `bezout_n`, al combinar el mcd acumulado con el siguiente elemento, los coeficientes anteriores se multiplican por el nuevo coeficiente de Bézout.

2.4 Potencias, inversas y función de Euler

`potencia_mod_p` usa exponenciación binaria. Recorre los bits del exponente, eleva la base al cuadrado en cada paso y multiplica el resultado por las potencias base$^{2^i}$ cuyo bit vale 1, reduciendo módulo p en cada operación. Esto es válido porque $a \equiv a'$ y $b \equiv b' \pmod p$ implican $ab \equiv a'b' \pmod p$. El coste es logarítmico en el exponente. Si el exponente es negativo, se sustituye la base por su inversa. Un módulo negativo se trata como su valor absoluto, y el módulo 0 y la expresión 0^0 lanzan `ZeroDivisionError`.

`inversa_mod_p(n,p)` existe si y solo si $\gcd(n, p) = 1$. En ese caso la identidad de Bézout da $nx + py = 1$, luego $nx \equiv 1 \pmod p$, y se devuelve $x \bmod p$. Si n no es invertible, se lanza `ZeroDivisionError`.

`euler` aplica la fórmula del producto a la factorización $n = \prod p_i^{k_i}$:

$$\varphi(n) = \prod_i p_i^{k_i - 1}(p_i - 1),$$

que se obtiene de que φ es multiplicativa y de que $\varphi(p^k) = p^{k-1}(p-1)$. Esto evita contar coprimos uno a uno.

2.5 Símbolo de Legendre

Para un primo impar p, el criterio de Euler afirma que

$$\left(\frac{n}{p}\right) \equiv n^{(p-1)/2} \pmod{p},$$

de modo que la potencia vale 1 si n es un residuo cuadrático no nulo y $p-1 \equiv -1$ si no lo es. `legendre` reduce n módulo p, devuelve 0 si $p \mid n$ y, en otro caso, calcula la potencia con `potencia_mod_p`.

2.6 Sistemas de congruencias

`resolver_sistema_congruencias` resuelve $a_i x \equiv b_i \pmod{p_i}$ sin exigir que los módulos sean coprimos dos a dos. Se procede en dos pasos:

1. **Reducción de cada ecuación.** $ax \equiv b \pmod{p}$ tiene solución si y solo si $d = \gcd(a,p)$ divide a b. En ese caso equivale a $x \equiv (a/d)^{-1}(b/d) \pmod{p/d}$. Si alguna ecuación no cumple la condición, el sistema es incompatible.
2. **Combinación sucesiva.** Supongamos que las ecuaciones ya tratadas equivalen a $x \equiv r \pmod{M}$ y que la siguiente es $x \equiv c \pmod{p}$. Toda solución de la primera es $x = r + Mk$, así que hay que resolver $Mk \equiv c - r \pmod{p}$. Con $g = \gcd(M,p)$, la ecuación tiene solución si y solo si $g \mid c-r$. Escribiendo $M = gu$ y $p = gv$, con u y v coprimos, se obtiene $k \equiv u^{-1}\frac{c-r}{g} \pmod{v}$. La nueva congruencia es $x \equiv r + Mk \pmod{Mv}$, donde $Mv = \operatorname{mcm}(M,p)$.

Cuando los módulos son coprimos, $g = 1$ en todos los pasos y el procedimiento coincide con el teorema chino del resto. El resultado es un par (r,m) con $0 \le r < m$, y las soluciones son todos los enteros congruentes con r módulo m. Los módulos negativos se tratan como su valor absoluto.

2.7 Raíces cuadradas: algoritmo de Cipolla

`raiz_mod_p(n,p)` busca un x con $x^2 \equiv n \pmod{p}$, siendo p primo. Los casos $p = 2$ y $n \equiv 0$ son inmediatos. Si $\left(\frac{n}{p}\right) = -1$, la raíz no existe y se lanza `IncompatibleEquationError`. En otro caso se aplica el algoritmo de Cipolla:

1. **Elegir t.** Se prueban $t = 0, 1, 2, \ldots$ hasta que $u = t^2 - n$ no sea residuo cuadrático, es decir, hasta que $\left(\frac{u}{p}\right) = -1$. Aproximadamente la mitad de los valores sirven, así que bastan muy pocos intentos.
2. **Ampliar el cuerpo.** Como u no tiene raíz en $\mathbb{F}_p$, se añade un elemento ω con $\omega^2 = u$. Se trabaja así con los números de la forma $a + b\omega$, que se multiplican como

$$(a + b\omega)(c + d\omega) = (ac + bd\,u) + (ad + bc)\,\omega.$$

3. **Calcular la raíz.** La raíz es $x = (t + \omega)^{(p+1)/2}$, que se calcula con exponenciación binaria sobre pares (a,b).

Que x sea una raíz se justifica en tres líneas. Por el criterio de Euler, $u^{(p-1)/2} = -1$, luego

$$\omega^p = \omega\,(\omega^2)^{(p-1)/2} = \omega\,u^{(p-1)/2} = -\omega.$$

Elevar a p respeta las sumas en característica p y $t^p = t$ (pequeño teorema de Fermat), así que $(t+\omega)^p = t - \omega$. Por tanto

$$x^2 = (t+\omega)^{p+1} = (t+\omega)^p\,(t+\omega) = (t-\omega)(t+\omega) = t^2 - u = n.$$

Como n es un residuo cuadrático, sus dos raíces están en $\mathbb{F}_p$: la parte en ω de x es nula y la función devuelve la parte entera. La otra raíz es $p - x$.

2.8 Ecuaciones de segundo grado

`ecuacion_cuadratica(a,b,c,p)`, con p primo impar y $a \not\equiv 0 \pmod{p}$, aplica la fórmula

$$x \equiv (-b \pm \sqrt{D})\,(2a)^{-1} \pmod{p}, \qquad D = b^2 - 4ac.$$

Si $D \equiv 0$ hay una raíz doble. Si $\left(\frac{D}{p}\right) = -1$ no hay solución y se lanza `IncompatibleEquationError`. En otro caso se calcula $\sqrt{D}$ con `raiz_mod_p`. Si $a \equiv 0 \pmod{p}$, la ecuación no es de segundo grado y también se lanza `IncompatibleEquationError`.

3 Intérprete IMAT-LAB

IMAT-LAB da acceso a la librería mediante comandos de texto:

Comando	Resultado
`primo(n)`	`Sí` o `No`.
`primos(a,b)`	Primos de $[a, b)$, separados por comas; el intervalo puede darse en cualquier orden.
`factorizar(n)`	Pares `primo: exponente`.
`mcd(a,b,...)`	Máximo común divisor de dos o más enteros.
`coprimos(a,b)`	`Sí` o `No`.
`pow(a,b,p), inv(n,p)`	$a^b \bmod p$ y $n^{-1} \bmod p$.
`euler(n), legendre(n,p)`	$\varphi(n)$ y $\left(\frac{n}{p}\right)$.
`resolverSistema([a;b;p],...)`	Solución en la forma `r (mod m)`.
`raiz(n,p), ecCuadratica(a,b,c,p)`	Soluciones ordenadas y sin repetir.

Si una línea no corresponde a ningún comando, o sus argumentos no son enteros, la salida es `NOP`. Si el comando es válido pero no tiene solución (una inversa que no existe, un sistema incompatible, un no residuo, etc.), la salida es `NE`.

Modos de uso.

- `python imatlab.py`: modo interactivo. Muestra la lista de comandos y procesa uno tras otro hasta que se introduce una línea vacía.
- `python imatlab.py "comando"`: ejecuta un único comando.
- `python imatlab.py entrada.txt`: ejecuta un fichero de comandos, uno por línea, y escribe los resultados en pantalla.
- `python imatlab.py entrada.txt salida.txt`: escribe los resultados en un fichero.

Diseño. Cada comando tiene una expresión regular que reconoce su sintaxis y captura los argumentos, y una función propia que los convierte a enteros, llama a la librería y da formato a la salida. Una lista asocia cada expresión con su función, en el orden en que se prueban. El orden importa en un caso: `mcd(a,b)` se comprueba antes que `mcd(a,b,...)`, porque ambas expresiones aceptan dos argumentos. El tratamiento de errores está centralizado en `procesar_linea`: un `ValueError` se traduce en `NOP`, y `ZeroDivisionError` o `IncompatibleEquationError` en `NE`. Gracias a esta separación, añadir un comando consiste en escribir su expresión y su función.

4 Criptosistema RSA

4.1 Generación de claves

`generar_claves(min_primo, max_primo)` sigue el esquema clásico:

1. Obtiene con `lista_primos` los primos del intervalo y elige al azar dos distintos, p_1 y p_2. Si el intervalo no contiene dos primos distintos, lanza `ValueError`.
2. Calcula $n = p_1 p_2$ y $\varphi(n) = (p_1 - 1)(p_2 - 1)$, aplicando que φ es multiplicativa y que $\varphi(p) = p - 1$.
3. Elige al azar un exponente $e \in [2, \varphi(n) - 1]$ coprimo con $\varphi(n)$, comprobado con `coprimos`.
4. Calcula $d \equiv e^{-1} \pmod{\varphi(n)}$ con `inversa_mod_p`.

La clave pública es (n, e) y la privada d. Cifrar es calcular $c = m^e \bmod n$ y descifrar, $m = c^d \bmod n$. Como $ed \equiv 1 \pmod{\varphi(n)}$, se cumple $m^{ed} \equiv m \pmod{n}$ para todo $0 \le m < n$: por el teorema de Euler si m es coprimo con n y, en general, aplicando el teorema chino del resto a p_1 y p_2.

¿Se pueden generar claves grandes (más de 20 dígitos) en un tiempo razonable? No. Para que n tenga 20 dígitos, los primos deben tener unos 10, y `generar_claves` construye la lista completa de primos del intervalo comprobando cada número con divisiones de prueba. El coste crece aproximadamente como $(b - a)\sqrt{b}$: en una ejecución de prueba, listar los primos de $[10^6, 2 \cdot 10^6)$ ya costó unos 3 segundos, y un intervalo de miles de millones de números es inabordable. Una implementación práctica generaría candidatos aleatorios del tamaño deseado y comprobaría cada uno con un test probabilístico como Miller–Rabin, sin construir ninguna lista.

4.2 Padding

Sin padding, RSA es determinista: un mismo mensaje produce siempre el mismo cifrado. Como el texto se cifra carácter a carácter, todas las apariciones de una letra producen el mismo número, lo que permite el ataque de la sección 6.2. El padding de la práctica rompe esta regularidad añadiendo k cifras decimales aleatorias al final del mensaje antes de cifrar:

$$m' = m \cdot 10^k + r, \qquad 0 \le r < 10^k.$$

Al descifrar se recupera m' y se eliminan las últimas k cifras: $m = \lfloor m'/10^k \rfloor$. Para que el proceso sea correcto debe cumplirse $m' < n$, es decir, $m < n \cdot 10^{-k}$. Con k cifras de padding, cada carácter puede cifrarse de 10^k formas distintas. Se trata de un padding didáctico: los esquemas reales, como OAEP, son aleatorios y además resisten otros ataques.

4.3 Cifrado de cadenas

`codificar_cadena` convierte cada carácter en su código Unicode (`ord`) y `cifrar_cadena_rsa` cifra cada código por separado, con el padding indicado. El resultado es una lista de enteros. El descifrado aplica el proceso inverso; si algún valor descifrado no corresponde a un carácter válido, se lanza `ValueError`.

5 Programas de usuario

5.1 Registro de usuarios

`registrarusuario.py` pide el nombre del usuario, los extremos del intervalo en el que se buscarán los primos y el número de cifras de padding. El programa comprueba que:

- los extremos son enteros y están ordenados ($a < b$);
- el intervalo contiene al menos dos primos distintos (si no, informa del error y vuelve a pedir el intervalo);
- el padding es un entero no negativo.

Después genera las claves con `generar_claves` y crea, si no existe, la carpeta `Usuarios` con dos ficheros: `pub_`⟨*nombre*⟩`.txt`, con n, e y el número de cifras de padding (una línea cada uno), y `priv_`⟨*nombre*⟩`.txt`, con d.

5.2 Criptochat

`python criptochat.py usuario1 usuario2` abre un chat en el que `usuario1` se comunica con `usuario2`. Al arrancar, comprueba que existen las claves pública y privada de `usuario1` y la clave pública de `usuario2`. Después ofrece tres opciones:

- **C (cifrar):** cifra un texto con la clave pública y el padding de `usuario2` y lo muestra como enteros separados por espacios.
- **D (descifrar):** descifra una secuencia de enteros con la clave privada de `usuario1`. Si la entrada no es una secuencia de enteros o no se puede decodificar, muestra un aviso y el chat continúa.
- **S (salir).**

Un mensaje que `usuario2` cifra para `usuario1` solo puede descifrarlo `usuario1`. Si se descifra con otra clave, el resultado son caracteres sin sentido, ya que el esquema no incluye ningún mecanismo para detectarlo.

6 Ataques a RSA

6.1 Ataque contra la clave

`romper_clave(n,e)` recupera la clave privada a partir de la pública invirtiendo el proceso de generación:

1. Calcula $\varphi(n)$ con `euler`, que factoriza n por división de prueba y obtiene así p_1 y p_2.
2. Calcula $d \equiv e^{-1} \pmod{\varphi(n)}$ con `inversa_mod_p`.

Con d, cualquier mensaje se descifra con `descifrar_cadena_rsa`. La seguridad de RSA depende de que factorizar n sea inviable. Aquí lo es solo para claves pequeñas: la división de prueba necesita del orden de $\sqrt{n}$ operaciones, de modo que un n de 13 cifras se factoriza en centésimas de segundo, mientras que uno de 20 cifras ya requiere del orden de 10^9 divisiones. Con los tamaños de clave reales (al menos 2048 bits) ni siquiera los mejores algoritmos de factorización conocidos son viables.

6.2 Ataque de texto claro elegido

`ataque_texto_elegido(cList,n,e)` descifra un mensaje cifrado sin padding usando solo la clave pública, sin calcular d. Aprovecha que el cifrado es determinista y se hace carácter a carácter: el atacante puede cifrar los mensajes que quiera con la clave pública, que es conocida por todos.

1. Cifra con (n, e) los códigos Unicode $0, 1, 2, \ldots$ y anota los que producen alguno de los valores del mensaje interceptado.
2. Se detiene cuando ha identificado todos los valores distintos del mensaje. Los textos habituales usan códigos bajos, así que esto ocurre tras unos pocos cientos de cifrados.
3. Sustituye cada valor cifrado por su carácter. Si algún valor no aparece entre los $1\,114\,112$ códigos posibles, el mensaje no se cifró sin padding con esa clave y se lanza `ValueError`.

Este ataque funciona aunque n sea enorme, porque no intenta factorizarlo. Lo que explota es el uso de RSA sin padding y carácter a carácter: con padding, un mismo carácter da lugar a cifrados distintos y la tabla deja de servir.

7 Limitaciones

- Es un trabajo didáctico y no debe usarse para proteger información: las claves son pequeñas, el texto se cifra carácter a carácter, el padding no es un esquema estándar y no hay autenticación ni control de integridad.
- La generación de claves construye la lista completa de primos del intervalo, por lo que no escala a claves grandes.
- `romper_clave` se basa en división de prueba y solo es viable para módulos pequeños.
- `raiz_mod_p` y `ecuacion_cuadratica` suponen que el módulo es primo y no lo comprueban.

8 Bibliografía

- Departamento de Matemática Aplicada, ICAI. *Números* y transparencias de RSA de la asignatura (material de clase).
- T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*.
- V. Shoup. *A Computational Introduction to Number Theory and Algebra*.
- D. H. Lehmer. *Euclid's Algorithm for Large Numbers*.
- K. Conrad. *The Miller–Rabin Test*.
- CP-Algorithms: *Integer factorization* y *Tests de primalidad*.
- Wikipedia: *Exponenciación modular*, *Test de primalidad de Miller–Rabin*, *Símbolo de Legendre*, *Cipolla's algorithm* y *RSA*.
- Wikilibros: *Implementación de algoritmos de teoría de números/Raíz cuadrada modular*.
- Universidad de Chile, apuntes de álgebra: `https://www.matematicas.ciencias.uchile.cl/juaco/section-17.html`.

- Blogs *El sendero de los primos* (primos de la forma $6k \pm 1$) y de M. García Larragán (exponenciación modular).
- Nanyang Technological University: guía de expresiones regulares. *Python Module of the Week*: módulo `io`.

Uso de inteligencia artificial

Durante las prácticas se utilizó la inteligencia artificial como apoyo para resolver dudas conceptuales sobre la teoría vista en clase, para buscar formas de mejorar la eficiencia del código (así se conocieron los algoritmos de Miller–Rabin y de Lehmer, que se estudiaron e implementaron) y para dudas puntuales de programación y redacción. El código y los cálculos se realizaron por los autores, y el contenido obtenido con ayuda de la IA se verificó con los conceptos de la asignatura.